

Fases de compilación

Santiago Javier Proaño Yépez

<2026-10-03 sáb>

Contents

1	Conceptos de la lengua	1
1.1	Léxico	1
1.2	Sintaxis	2
1.3	Semántica	2
2	Fases de la compilación	2
2.1	Fuente	2
2.2	Preprocesador	2
2.3	Análisis léxico	2
2.4	Análisis sintáctico	3
2.5	Análisis semántico	3
2.6	Generación de código intermedio	3
2.7	Optimización del código	4
2.8	Generación de código objeto	4

1 Conceptos de la lengua

Antes de entrar en que hace la compilación tenemos que entender los 3 conceptos claves.

1.1 Léxico

Son todas las palabras correctamente escritas. Viéndolo desde el mundo real un error de léxico seria “El gatx come pescado” aquí gatx no existe en el lenguaje, por lo que esta mal lexicamente.

1.2 Sintaxis

Es la gramática de la oración, básicamente verifica como están ordenadas las palabras en sujeto verbo y predicado. Pueden estar bien escritas pero no entenderse nada. Por ejemplo: “Pescado el come el gato” todas las palabras existen pero el orden es imposible.

1.3 Semántica

Es el significado de la oración, verifica si tiene sentido lo que se dice. Una oración puede tener todas las palabras existentes, tener sentido en orden, pero no tener un significado correcto. Por ejemplo: “La mesa bebe café”. Todo existe, el orden esta bien, pero no tiene un significado correcto.

2 Fases de la compilación

Compilar tiene 8 bloques importantes, y se verán a través de un ejemplo:

2.1 Fuente

Programador diseña y escribe el código en un lenguaje de alto nivel, es decir que usa lenguaje cercano al del ser humano.

```
posicion = inicial + velocidad * 60
```

2.2 Preprocesador

Junta todo el programa en uno solo y lo deja listo para empezar a compilar

2.3 Análisis léxico

Lee lo escrito en el programa de izquierda a derecha, carácter por carácter, definiendo dos cosas

- Cuando una palabra es valida, por ejemplo **velocidad**, se le clasifica como un **lexema**.
- A dicho lexema se lo verifica y se agrega un identificador, por ejemplo = seria **ASIG** estos son los **tokens**

Verifica que cada palabra este bien escrita.

```

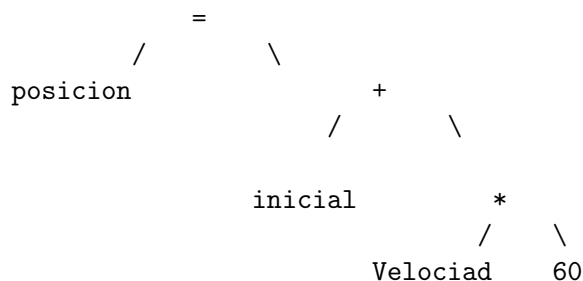
posicion -> ID
= -> ASIG
inicial -> ID
+ -> SUMA
velocidad -> ID
(*) -> MLUT
60 -> NUM

```

2.4 Análisis sintáctico

Se construye un árbol sintáctico con los tokens siguiendo una regla de oro “Se debe considerar que lo que esta en la parte superior es lo ultimo en ejecutar y lo que esta en la parte inferior es lo primero en ejecutar”.

Verifica que el orden este correcto. Entonces recorremos de izquierda a derecha y analizamos cual va ultimo y así



2.5 Análisis semántico

Comprueba que las operaciones tengan sentido con los tipos de datos que es manejan. En este caso posición, inicial y velocidad están definidas como tipo `double` y 60 es entero entonces toca hacerle de tipo `double` como 60.0

Aquí vemos que entra la tabla de símbolos

Nombre	Clase	Tipo	Alcance
posición	variable	real	local
inicial	variable	real	local
velocidad	variable	real	local

2.6 Generación de código intermedio

Convierte el árbol sintáctico en instrucciones simples, operando linea por linea, este no depende de ningún lenguaje de fuente ni maquina de destino.

Se lo hace en código de tres direcciones Códigos de direcciones

```
t1 = intToReal(60)
t2 = t1 * velocidad
t3 = inicial + t2
posicion = t3
```

2.7 Optimización del código

Básicamente toma el código intermedio y le disminuye las instrucciones sin cambiar el resultado del programa. En el caso de ejemplo el `intToReal(60)` se lo puede hacer en tiempo de compilación así que no hace falta ejecutarle cada vez. Esto se le llama constant folding

```
t2 = velocidad * 60.0
t3 = inicial + t2
posicion = t3
```

2.8 Generación de código objeto

Cambia y traduce el código intermedio ya optimizado en instrucciones de maquina, o ensamblador, esta fase ya depende del hardware.

```
LOADF velocidad, R2
MULTF #60.0, R2
LOADF inicial, R1
ADDF R2,R1
STOREF R1, posicion
```

En este código ilustrativo tenemos que Rx son los registros del CPU donde están guardándose las variables.